DEVPILOT AI

Spring Boot REST Guidelines

Practical API engineering for Java 21 and Spring Boot projects

A retrieval-friendly engineering reference for designing, implementing, validating, and reviewing maintainable REST APIs. It focuses on project-aware changes and does not require a particular authentication, persistence, or deployment design.

Document	Value
Version context	Java 21; Spring Boot 4.1.1 as specified by the project context
Audience	Java engineers, code reviewers, Research Agent and Coding Agent
Core flow	HTTP Controller → Application Service → Repository or external port
Normative terms	MUST / SHOULD / MAY
RAG format	Literal Heading: and Section: prefixes in extracted text

Compatibility rule: Use APIs and properties supported by the actual dependency versions in the repository. Inspect the POM and existing configuration; do not copy a version-specific snippet blindly.

Heading: Document Navigation — Contents and usage

The literal prefixes below and throughout the document are included as ordinary PDF text so the existing section-aware RAG pipeline can identify major topics and focused subsections.

1. Scope and implementation principles
2. Maven and Spring Boot project structure
3. Package responsibilities and dependency direction
4. Controller → Service → Repository flow
5. Dependency injection and configuration
6. REST resource design and HTTP methods
7. Status codes and response contracts
8. DTOs, entities, and mapping
9. Jakarta Bean Validation
10. Centralized exception handling
11. Externalized configuration and secrets
12. Logging and error hygiene
13. Pagination, sorting, and performance
14. Dependency selection
15. Step-by-step endpoint workflow
16. Complete endpoint example
17. Incorrect versus corrected patterns
18. Common mistakes
19. Coding-agent workflow and review checklist

> **Apply in context:** Preserve the existing project's architecture and public contracts. The patterns here are defaults, not a reason to restructure unrelated modules.

Heading: Scope and Implementation Principles

Section: Purpose and audience

This guide supports implementation and review of Java REST APIs built with Spring Boot. It covers project layout, request handling, validation, error responses, configuration, persistence boundaries, and endpoint implementation workflow. It does not prescribe a specific authentication model, database, API versioning strategy, or deployment topology.

Section: MUST, SHOULD, and MAY

Term	Meaning	Example
MUST	Required unless a documented project constraint justifies an exception.	MUST validate untrusted request input.
SHOULD	Strong default; exceptions need a clear reason.	SHOULD use DTOs rather than expose persistence entities.
MAY	Optional technique when it solves a real need.	MAY add pagination to a large collection endpoint.

Section: Repository-first implementation

- MUST inspect pom.xml, Java release, Spring Boot dependency management, package layout, DTOs, exception handlers, security configuration, and nearby endpoints before editing.
- MUST preserve existing API contracts unless the requirement explicitly changes them.
- SHOULD reuse the project's response envelope, naming convention, mapper, validation style, and error format.
- MUST not add a dependency merely because an example uses it; check whether the capability already exists.
- MUST distinguish repository facts from assumptions, and ask for clarification when ambiguity affects external behavior.

Heading: Maven and Spring Boot Project Structure

Section: Typical Maven layout

```
src/
  main/
    java/com/example/app/
      api/                  # HTTP controllers and request/response DTOs
      service/              # application use cases and orchestration
      domain/               # domain model and business rules (if applicable)
      repository/           # persistence adapters or Spring Data repositories
      mapper/               # explicit DTO/domain mapping (if useful)
      config/               # configuration and framework wiring
      exception/            # domain/application exceptions and API advice
      integration/          # external-service adapters (optional)
      Application.java
    resources/
      application.yml
  test/
    java/com/example/app/
      api/
      service/
      repository/
    resources/
```

This is a starting point, not a required package taxonomy. Small services may use fewer packages; larger systems may organize by feature (for example, billing.api, billing.service, billing.repository). Follow the repository's established structure and avoid moving code without a requirement.

Section: Maven and dependency management

A Spring Boot Maven project commonly inherits dependency and plugin management from the Spring Boot parent or imports its dependency BOM. Use one coherent version-management approach. Do not add versions to individual dependencies when the parent/BOM is expected to manage them unless the project deliberately overrides that version.

```
&lt;!-- Illustrative only: use the project's existing parent/BOM and versions. --&gt;
&lt;dependency&gt;
    &lt;groupId&gt;org.springframework.boot&lt;/groupId&gt;
    &lt;artifactId&gt;spring-boot-starter-web&lt;/artifactId&gt;
&lt;/dependency&gt;
```

The project's specified Spring Boot version is 4.1.1. Verify the actual POM and the available release documentation before relying on any feature or dependency coordinate. The examples in this guide emphasize stable Spring MVC and Jakarta Validation concepts; they are not a claim that every snippet was compiled against the project's full dependency graph.

Heading: Package Responsibilities and Dependency Direction

Section: Controller

A controller is the HTTP boundary. It maps routes and verbs, binds request data, invokes validation, delegates to an application service, and translates the result to an HTTP response. Controllers SHOULD remain thin: do not put database queries, business workflows, or large domain decisions in controller methods.

Section: Service

A service coordinates a use case and enforces application-level business rules. It may call repositories and external-service ports, define transaction boundaries when persistence is involved, and return a domain or application result. A service should not depend on servlet request objects unless the use case genuinely requires transport-specific context.

Section: Repository

A repository abstracts persistence access. It should expose operations meaningful to its caller and avoid embedding HTTP response decisions. Use Spring Data repositories, JDBC, or another persistence approach based on the project and query requirements; this document does not require a particular database technology.

Section: DTO, entity, mapper, configuration, and exception responsibilities

Component	Responsibility	Avoid
Request DTO	Shape and validate incoming API data.	Using it as a JPA entity by convenience.
Response DTO	Expose a stable API representation.	Returning database entities with lazy relations.
Entity/domain model	Represent persistence or business state according to project design.	Coupling all domain rules to HTTP serialization.
Mapper	Translate between types when mapping is non-trivial or repeated.	Adding a mapping framework for two trivial fields without need.
Configuration	Declare framework beans and externalized wiring.	Hard-coding environment-specific secrets.
Exception advice	Map known failures to stable HTTP error responses.	Returning stack traces or catching every failure as success.

Section: Dependency direction

The usual request path is controller → service → repository or integration adapter. Keep dependencies flowing toward the component that owns the relevant behavior. A repository should not call a controller. A service should not construct controllers or depend on HTTP serialization classes just to make a use case work.

```
HTTP request
    |
    v
Controller  -- bind, validate, translate HTTP
    |
    v
Application Service  -- use case and business rules
    |
    +----&gt; Repository / persistence port
    |
    +----&gt; External service adapter
    |
    v
Result -&gt; response DTO -&gt; HTTP response
```

Heading: Controller → Service → Repository Flow

Section: Thin controller example

```
@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping("/{id}")
    public ProductResponse getById(@PathVariable UUID id) {
        return productService.getById(id);
    }
}
```

The example assumes ProductService exposes a method that returns a response DTO. In a larger codebase, mapping may be performed in the service or a dedicated mapper. The important boundary is that the controller does not own persistence or domain logic.

Section: Service and repository contracts

```
public interface ProductRepository {
    Optional&lt;Product&gt; findById(UUID id);
}

@Service
public class ProductService {
    private final ProductRepository productRepository;
    private final ProductMapper productMapper;

    public ProductService(ProductRepository productRepository,
                          ProductMapper productMapper) {
        this.productRepository = productRepository;
        this.productMapper = productMapper;
    }

    public ProductResponse getById(UUID id) {
        Product product = productRepository.findById(id)
                .orElseThrow(() -&gt; new ProductNotFoundException(id));
        return productMapper.toResponse(product);
    }
}
```

A custom repository interface and mapper are illustrative. If the project already uses Spring Data interfaces or direct DTO projections, adapt the example rather than introducing unnecessary layers.

Heading: Dependency Injection and Configuration

Section: Constructor injection

Constructor injection makes required dependencies explicit, allows fields to be final, and supports straightforward unit tests. For a single constructor, Spring can generally use it without an explicit @Autowired annotation. Avoid field injection in application code because it hides dependencies and makes direct construction/testing harder.

```
@Service
public class CatalogService {
    private final CatalogRepository repository;

    public CatalogService(CatalogRepository repository) {
        this.repository = Objects.requireNonNull(repository);
    }
}
```

Section: Avoid service locator patterns

A component should declare what it needs rather than repeatedly asking the application context for beans. Do not call ApplicationContext.getBean() as a substitute for ordinary dependency injection unless dynamic lookup is a genuine framework requirement.

Section: Bean scope and mutable state

Spring-managed singleton beans are shared by default. Services and controllers SHOULD be stateless with respect to per-request data. Do not store request-specific values in mutable instance fields. If shared state is needed, define and test its concurrency behavior explicitly.

Heading: REST Resource Design and HTTP Methods

Section: Resource-oriented paths

- SHOULD name resources with nouns, typically plural collections such as /api/orders and /api/orders/{orderId}.
- SHOULD use path variables to identify a resource and query parameters for filtering, sorting, pagination, or optional selection.
- MUST use consistent naming and versioning conventions already adopted by the API.
- SHOULD avoid action verbs in paths when the action is naturally represented by an HTTP method.
- MAY use a command-style subresource for a domain operation that is not naturally CRUD, when documented and consistent.

Method	Typical intent	Semantics to respect
GET	Read a resource or collection.	Safe; should not change application state.
POST	Create a resource or execute a non-idempotent command.	Often returns 201 Created for successful creation.
PUT	Replace a resource representation.	Normally idempotent; define replacement semantics.
PATCH	Partially update a resource.	Define patch format and behavior for omitted/null fields.
DELETE	Remove a resource or relationship.	Should be idempotent in its externally observable effect where practical.

Section: Path and query examples

```
GET    /api/orders?status=PAID&amp;page=0&amp;size=20
GET    /api/orders/5a4d...
POST   /api/orders
PUT    /api/orders/5a4d...
PATCH  /api/orders/5a4d...
DELETE /api/orders/5a4d...
```

Use stable identifiers and document whether collection endpoints are paginated. Do not expose internal database table names or implementation details in public routes.

Heading: HTTP Status Codes and Response Contracts

Section: Status-code selection

Status	Use
200 OK	Successful read or update with a response body.
201 Created	A resource was created; include its representation and/or Location header where useful.
202 Accepted	Work was accepted for asynchronous processing but is not complete.
204 No Content	Successful operation with no response body.
400 Bad Request	Malformed request or invalid request data, according to the API contract.
401 Unauthorized	Authentication is missing or invalid.
403 Forbidden	The caller is authenticated but not allowed to perform the operation.
404 Not Found	The requested resource does not exist or is intentionally not disclosed.
409 Conflict	The request conflicts with current resource state or a uniqueness/state constraint.
422 Unprocessable Content	Optional convention for syntactically valid content that fails semantic validation; use consistently if adopted.
500 Internal Server Error	Unexpected server failure; never expose internals in the response.

Section: Response envelope consistency

Choose a consistent API response contract. Some projects return resource DTOs directly; others use a standard envelope with data, error, and metadata fields. Neither shape is universally required. The agent MUST reuse the existing contract unless asked to change it, because clients may rely on the exact JSON structure.

```
// Illustrative error response; match the project's existing contract.
public record ApiError(
        String code,
        String message,
        String path,
        Instant timestamp
) {}
```

For new APIs, Problem Details can be appropriate when supported by the framework and API contract. Do not mix multiple error formats without a clear migration plan.

Heading: Request and Response DTOs; Entities and Mapping

Section: Why DTOs matter

DTOs decouple the public API from persistence schema and internal domain structure. They let the application control which fields clients can submit or see, support API evolution, and keep validation rules at the request boundary. Returning JPA entities directly can leak internal fields, trigger lazy loading during serialization, or create accidental recursive JSON graphs.

```
public record CreateProductRequest(
        @NotBlank String name,
        @PositiveOrZero BigDecimal price
) {}

public record ProductResponse(
        UUID id,
        String name,
        BigDecimal price
) {}
```

Section: Mapping rules

- MUST not bind untrusted request JSON directly to a persistence entity if that exposes protected fields or bypasses domain invariants.
- SHOULD keep request DTOs separate from response DTOs when clients can set only a subset of fields.
- SHOULD make mapping explicit and easy to review; use a mapper library only when it reduces meaningful repetition.

- MUST define how missing fields and explicit nulls behave for partial updates.
- SHOULD avoid exposing password hashes, internal flags, persistence versions, or internal relationship graphs in API responses.

Section: Records as DTOs

Java records are suitable for simple DTOs whose components are fixed. They do not validate automatically: Jakarta Bean Validation annotations and controller validation must be configured correctly, and domain invariants may still need enforcement in the service/domain layer.

Heading: Jakarta Bean Validation

Section: Validate at the HTTP boundary

Use Jakarta Bean Validation constraints for request shape and basic value rules. In Spring MVC, annotate a request body parameter with @Valid or the appropriate validation annotation used by the project, and ensure a validation provider is present. The precise dependency and method-validation behavior should be confirmed against the actual Spring Boot 4.1.1 dependency graph.

```
@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity&lt;ProductResponse&gt; create(
            @Valid @RequestBody CreateProductRequest request) {
        ProductResponse created = service.create(request);
        URI location = URI.create("/api/products/" + created.id());
        return ResponseEntity.created(location).body(created);
    }
}
```

Section: Common constraints

Constraint	Typical purpose
@NotNull	Value must not be null.
@NotBlank	String must not be null and must contain a non-whitespace character.
@Size	Bound string, collection, or supported value size.
@Email	Check a string for email-like format; not proof that an address exists.
@Positive / @PositiveOrZero	Enforce numeric sign constraints.
@Min / @Max	Bound supported numeric values.

Section: Validation boundaries

Request validation does not replace domain validation. A service can be called by a scheduled job, message consumer, or another internal component without going through the controller. Enforce business invariants at the layer that owns them; avoid duplicating every transport-specific constraint in every layer.

Heading: Centralized Exception Handling

Section: Use ControllerAdvice

Centralized exception handling maps known exceptions to stable HTTP responses and avoids repeated try/catch blocks in every controller. The handler should catch exceptions it understands, log unexpected failures safely, and avoid leaking stack traces, SQL details, file paths, or credentials to clients.

```
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
```

```
    public ResponseEntity&lt;ApiError&gt; handleNotFound(
            ProductNotFoundException ex,
            HttpServletRequest request) {
        ApiError error = new ApiError(
                "PRODUCT_NOT_FOUND",
                "Product was not found",
                request.getRequestURI(),
                Instant.now()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity&lt;ApiError&gt; handleValidation(
            MethodArgumentNotValidException ex,
            HttpServletRequest request) {
        ApiError error = new ApiError(
                "VALIDATION_FAILED",
                "One or more request fields are invalid",
                request.getRequestURI(),
                Instant.now()
        );
        return ResponseEntity.badRequest().body(error);
    }
}
```

Imports and exact framework method signatures should be checked against the project's Spring Boot version. The example intentionally avoids returning raw binding exception messages, which may be noisy or reveal implementation details.

Section: Unexpected errors

An unexpected exception should produce a generic 500 response and a server-side diagnostic event with a correlation identifier where available. Do not turn all exceptions into 400 responses: server bugs and infrastructure failures are not necessarily client errors.

- MUST preserve a stable error schema.
- MUST not return exception stack traces, SQL, secrets, or internal class names to clients.
- SHOULD map domain failures to appropriate status codes at one boundary.
- SHOULD avoid logging the same exception repeatedly at every layer.
- MAY include field-level validation details when the API contract allows it and the messages are safe.

Heading: Configuration and Secret Management

Section: Externalize environment-specific values

Keep ports, service URLs, feature flags, and other environment-specific settings in configuration rather than scattering literals throughout code. Use application.yml and Spring's property binding patterns already established in the project. Validate required configuration at startup where possible.

```
# Illustrative configuration; values are environment-specific.
app:
  catalog:
    base-url: ${CATALOG_BASE_URL}
    connect-timeout: ${CATALOG_CONNECT_TIMEOUT:2s}
```

Section: Secrets

- MUST not commit passwords, tokens, API keys, private keys, or production credentials.
- SHOULD inject secrets through the deployment environment or an approved secret manager.
- MUST not print resolved secrets in startup logs or exception messages.
- SHOULD use typed configuration properties for groups of related settings when it improves validation and maintainability.
- MUST distinguish a missing required property from a safe default; do not silently use a dummy production credential.

Section: Configuration example

```
@ConfigurationProperties(prefix = "app.catalog")
public record CatalogProperties(
        URI baseUrl,
        Duration connectTimeout
) {
    public CatalogProperties {
        Objects.requireNonNull(baseUrl, "baseUrl");
        Objects.requireNonNull(connectTimeout, "connectTimeout");
        if (connectTimeout.isNegative() || connectTimeout.isZero()) {
            throw new IllegalArgumentException("connectTimeout must be positive");
        }
    }
}
```

Enable configuration-property scanning or registration according to the existing application setup. Property names and duration conversion should be verified against the actual framework configuration.

Heading: Logging and API Error Hygiene

Section: Useful request logging

- SHOULD log meaningful lifecycle events and failures with safe identifiers.
- MUST not log authorization headers, cookies, passwords, tokens, or full sensitive request bodies.
- SHOULD avoid logging every successful request at INFO if volume is high; use the project's established access-log strategy.
- SHOULD include a correlation or trace identifier when the application already supports one.
- MUST sanitize untrusted values if they could forge log lines.

Section: Validation and exception logs

Expected validation errors often do not need ERROR-level stack traces. Unexpected failures should be logged at the layer with enough context to diagnose them, without duplicate stack traces. Client-facing messages should remain stable and safe; detailed diagnostic data belongs in controlled server logs.

Heading: Pagination, Sorting, and Performance

Section: Paginate unbounded collections

Collection endpoints should not return an unbounded number of rows when data can grow substantially. Pagination protects memory, database load, and response latency. Choose offset pagination for straightforward page navigation when acceptable; consider cursor/keyset pagination for large, frequently changing datasets when stable traversal and performance justify the extra design work.

```
// Illustrative Spring Data style; adapt to the repository's persistence model.
@GetMapping
public Page&lt;ProductResponse&gt; list(
        @RequestParam(defaultValue = "0") @Min(0) int page,
        @RequestParam(defaultValue = "20") @Min(1) @Max(100) int size) {
    return productService.list(PageRequest.of(page, size));
}
```

Method-parameter validation can require method validation configuration and version-specific setup. Alternatively, validate parameters explicitly or use the project's established mechanism. Do not assume the annotations alone enforce every constraint.

Section: Sorting and query limits

- MUST validate or allowlist externally supplied sort fields; never concatenate arbitrary user input into SQL/order expressions.
- SHOULD impose a reasonable maximum page size.
- SHOULD request only fields needed by the response where the persistence layer supports projections.
- SHOULD inspect query counts and database access patterns before adding caching.
- MUST avoid N+1 query patterns when returning related data at scale.
- MAY use cursor pagination, projections, caching, or asynchronous processing when measurements and access patterns justify them.

Section: Performance measurement

Avoid speculative micro-optimizations. Measure slow endpoints using representative workloads, query plans, metrics, and traces. Add indexes based on real query patterns and validate their write/storage cost. Avoid returning full entities or loading relationships that the response does not need.

Heading: Dependency Selection Guidance

Section: Common dependencies and their purpose

Capability	Typical Maven coordinate	When justified
Spring MVC / REST	org.springframework.boot:spring-boot-starter-web	Needed for a conventional Spring MVC REST API; verify starter names in the project's Boot release.
Bean validation	org.springframework.boot:spring-boot-starter-validation	Needed when using Jakarta Bean Validation with a provider; check project version and dependency management.
Persistence	org.springframework.boot:spring-boot-starter-data-jpa	Only when using JPA/Hibernate repositories and ORM mapping.
JDBC access	org.springframework.boot:spring-boot-starter-jdbc	When using Spring JDBC and a JDBC driver rather than relying on JPA.
Testing	org.springframework.boot:spring-boot-starter-test	Common test dependency for Spring Boot tests; use test scope.
API documentation	springdoc-openapi modules or approved alternative	Optional; add only if the project requires generated OpenAPI documentation and compatibility is confirmed.

Section: Dependency rules

- MUST inspect the existing POM, dependency management, and transitive dependencies before adding a starter.
- MUST not add JPA, JDBC, mapping, API documentation, or validation dependencies just because they appear in this guide.
- SHOULD use the Boot parent/BOM to manage compatible versions rather than pinning unrelated versions.
- MUST verify third-party compatibility with Spring Boot 4.1.1 before adopting a library.
- SHOULD record why a new dependency is needed and what operational/configuration burden it adds.

Heading: Step-by-Step Workflow for a New REST Endpoint

Section: Implementation sequence

```
INPUT: endpoint requirement and existing repository

1. Read the API contract and identify method, path, request, response, status codes.
2. Inspect existing controllers, services, DTOs, repositories, exception advice, and tests.
3. Identify validation rules and authorization expectations from project configuration.
4. Define or reuse request/response DTOs.
5. Add service behavior and persistence calls at the correct layer.
6. Add controller mapping and HTTP status behavior.
7. Map known exceptions centrally to stable error responses.
8. Add tests for success, validation, not-found/conflict, and relevant edge cases.
9. Run focused tests and broader checks available in the environment.
10. Review diff for contract changes, secrets, unrelated edits, and missing error paths.
11. Report changed files, tests executed, outcomes, and limitations.
```

Section: Acceptance criteria template

- The route and HTTP method match the stated requirement.
- Request data is validated and unauthorized state changes are not permitted by the configured security model.
- Business logic is in an appropriate service/domain boundary.
- Response DTO and status code match the API contract.
- Known failures map to stable and safe errors.

- Tests cover success and meaningful failure paths.
- No unrelated architecture or dependency changes were introduced.

Heading: Complete Endpoint Example

Section: Use case and assumptions

The following compact example creates a product. It is intentionally illustrative: persistence implementation, package declarations, imports, security rules, transaction behavior, and error schema must match the actual application. The request DTO validates basic shape; the service should enforce domain rules and persist using the project's selected data-access strategy.

```
public record CreateProductRequest(
        @NotBlank @Size(max = 120) String name,
        @NotNull @PositiveOrZero BigDecimal price
) {}

public record ProductResponse(UUID id, String name, BigDecimal price) {}

public interface ProductRepository {
    Product save(Product product);
}

public record Product(UUID id, String name, BigDecimal price) {
    public Product {
        Objects.requireNonNull(id, "id");
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name must not be blank");
        }
        Objects.requireNonNull(price, "price");
        if (price.signum() &lt; 0) {
            throw new IllegalArgumentException("price must not be negative");
        }
    }
}

@Service
public class ProductService {
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    public ProductResponse create(CreateProductRequest request) {
        Product product = new Product(
                UUID.randomUUID(), request.name().trim(), request.price());
        Product saved = repository.save(product);
        return new ProductResponse(saved.id(), saved.name(), saved.price());
    }
}

@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity&lt;ProductResponse&gt; create(
            @Valid @RequestBody CreateProductRequest request) {
        ProductResponse response = service.create(request);
        URI location = URI.create("/api/products/" + response.id());
        return ResponseEntity.created(location).body(response);
    }
}
```

Section: Why this structure

- The controller owns HTTP binding and the 201 Created response.
- The request DTO describes what a client may submit, not the internal persistence schema.
- The service coordinates creation and repository interaction.

- The domain record enforces invariants even if constructed outside the controller.
- The repository contract keeps the service from depending on a specific database API.
- A production implementation still needs a concrete repository bean, centralized error handling, appropriate transaction semantics, and tests.

Heading: Incorrect Versus Corrected Patterns

Section: Business logic inside a controller

```
// Avoid: HTTP layer owns persistence and business decisions.
@PostMapping
Product create(@RequestBody Product product) {
    if (product.price().signum() &lt; 0) {
        throw new IllegalArgumentException();
    }
    return repository.save(product);
}

// Better: delegate to an application service.
@PostMapping
ResponseEntity&lt;ProductResponse&gt; create(
        @Valid @RequestBody CreateProductRequest request) {
    ProductResponse response = service.create(request);
    return ResponseEntity.created(URI.create("/api/products/" + response.id()))
            .body(response);
}
```

Section: Returning entities directly

```
// Avoid: entity fields/relations become part of the API by accident.
@GetMapping("/{id}")
ProductEntity get(@PathVariable UUID id) {
    return repository.findById(id).orElseThrow();
}

// Better: map to an explicit response DTO.
@GetMapping("/{id}")
ProductResponse get(@PathVariable UUID id) {
    return service.getById(id);
}
```

Section: Field injection

```
// Avoid: dependency is hidden and direct unit construction is awkward.
@Autowired
private ProductRepository repository;

// Prefer constructor injection.
private final ProductRepository repository;

public ProductService(ProductRepository repository) {
    this.repository = repository;
}
```

Section: Leaking exception details

```
// Avoid: internal details may reveal schema, paths, or secrets.
return ResponseEntity.internalServerError().body(ex.getMessage());

// Prefer a stable generic error and server-side diagnostics.
return ResponseEntity.internalServerError()
        .body(new ApiError("INTERNAL_ERROR",
                "An unexpected error occurred", path, Instant.now()));
```

Section: Unbounded list endpoint

```
// Avoid: result size can grow without a limit.
@GetMapping
List&lt;ProductResponse&gt; list() {
    return service.findAll();
}

// Prefer a bounded/paginated contract when the collection can grow.
@GetMapping
Page&lt;ProductResponse&gt; list(Pageable pageable) {
    return service.findPage(pageable);
}
```

Heading: Common Mistakes and Remediation

Mistake	Risk	Remediation
Controller calls database directly	Coupled HTTP and persistence logic; hard-to-test behavior.	Delegate to service and repository boundaries.
Entity used as request DTO	Clients may set protected/internal fields.	Use a request DTO and map explicitly.
One giant service	Low cohesion and many unrelated reasons to change.	Split by use case or cohesive domain responsibility.
Catch-all exception handler returns 400	Server faults appear to be client mistakes.	Map known exceptions specifically; return generic 500 for unexpected failures.
No maximum page size	Large responses can exhaust resources.	Set and validate a sensible maximum.
Hard-coded service URL or secret	Environment fragility and credential exposure.	Externalize configuration; keep secrets in approved secret storage.
New dependency without inspection	Version conflicts and unnecessary attack surface.	Inspect POM/BOM and justify the capability.
Method parameter annotations assumed to validate automatically	Constraints may not be enforced in the current setup.	Verify method-validation setup or validate explicitly.

Heading: Spring Boot REST API Review Checklist

Section: Contract and HTTP behavior

- Route, method, resource naming, and API versioning match project conventions.
- Status codes and response body shape are documented and consistent.
- Creation uses 201 Created and an appropriate Location header when suitable.
- No accidental breaking change to field names, nullability, or error format.

Section: Architecture and data boundaries

- Controller is thin; business rules live in an appropriate service/domain layer.
- Constructor injection is used for required dependencies.
- DTOs separate client-facing fields from persistence entities.
- Repository owns persistence operations and does not decide HTTP responses.
- No unrelated refactor or unnecessary dependency was introduced.

Section: Validation, errors, and operations

- External inputs have suitable validation and domain invariants remain protected.
- Known exceptions map centrally to stable status codes and error schemas.
- Unexpected errors do not expose stack traces or sensitive implementation details.
- Configuration is externalized and secrets are not committed or logged.
- Logs provide useful context without exposing tokens or personal data.

Section: Performance and testing

- Large collections are paginated or otherwise bounded.
- Sort fields and query parameters are validated/allowlisted.
- N+1 queries and unnecessary entity loading are considered where relevant.
- Tests cover success, validation failure, not-found/conflict, and important boundaries.
- Commands and test results in the implementation report are accurate.

End of PDF 2 — Spring Boot REST Guidelines. The next document in the sequence is PDF 3, Spring Security Guidelines.

Document filename: `spring-boot-rest-guidelines.pdf`